

```
(-flatten list) ;; Syntax
(-flatten-n N list) ;; Syntax

(-flatten '((5)))
(5)
(-flatten '((1 (2 3) (((4 (5)))))))
(1 2 3 4 5)

(-flatten-n 1 '((1 2) ((3 4) ((5 6)))))
(1 2 (3 4) ((5 6)))
```

The ‘-replace’ function replaces old items in the list with the new item. For example:

```
(-replace old new list) ;; Syntax

(-replace 1 "one" '(1 2 3 4 5))
("one" 2 3 4 5)
```

The ‘-replace-first’ function replaces the first occurrence of the old item in the list with the new item, while, the ‘-replace-last’ function replaces the last occurrence of the old item with the new item. A couple of examples to demonstrate the function application are given below:

```
(-replace-first old new list) ;; Syntax
(-replace-last old new list) ;; Syntax

(-replace-first 1 "one" '(1 2 1 2 1 2))
("one" 2 1 2 1 2)

(-replace-last 1 "one" '(1 2 1 2 1 2))
(1 2 1 2 "one" 2)
```

The ‘-insert-at’ function returns a list with the input element inserted at the specified position in the argument list. For example:

```
(-insert-at position element list) ;; Syntax

(-insert-at 1 2 '(1 3 4))
(1 2 3 4)
```

The ‘-replace-at’ function replaces an element in the specified position with the input element for the given argument list. In the following example, the element at index 1 is replaced with the number ‘2’.

```
(-replace-at position element list) ;; Syntax

(-replace-at 1 2 '(1 1 3 4))
(1 2 3 4)
```

The ‘-update-at’ function takes three arguments — a position, a function and an input list. It takes the element at the position, applies the function on the element replacing it with the function result, and returns the list. For example:

```
(-update-at position function list) ;; Syntax

(--update-at 1 (length it) '("thank" "you" "very" "much"))
("thank" 3 "very" "much")
```

The ‘-remove-at’ function returns a list with the element at the given position removed from the argument list. An example is given below:

```
(-remove-at position list) ;; Syntax

(-remove-at 2 '(1 2 3 4 5))
(1 2 4 5)
```

Reductions

In this section, we will go through functions that reduce a list to a single value. This is a reduction or merge function.

The ‘-reduce-from’ function takes three arguments — a function, an initial value and a list. It starts by applying the function to the initial value and the first element of the list. The result then becomes the new initial value or accumulator. The function is then applied to this new value and the second element of the list, and so on. For example:

```
(-reduce-from function initial-value list) ;; Syntax

(-reduce-from '- 100 '(10 20 30))
40
```

The function application occurs in the following order for the above example:

```
((- 100 10) 20 30) ; 100 - 10 = 90
((- 90 20) 30) ; 90 - 20 = 70
(- 70 30) ; 70 - 30 = 40
```

The ‘-reduce-r-from’ API is similar to the ‘-reduce-from’ function, except that the operation association occurs from the right-hand side instead of the left-hand side. An example is given below to illustrate the function application:

```
(-reduce-r-from function initial-value list) ;; Syntax

(-reduce-r-from '- 100 '(10 20 30))
-80
```

The function application evaluates in the following order